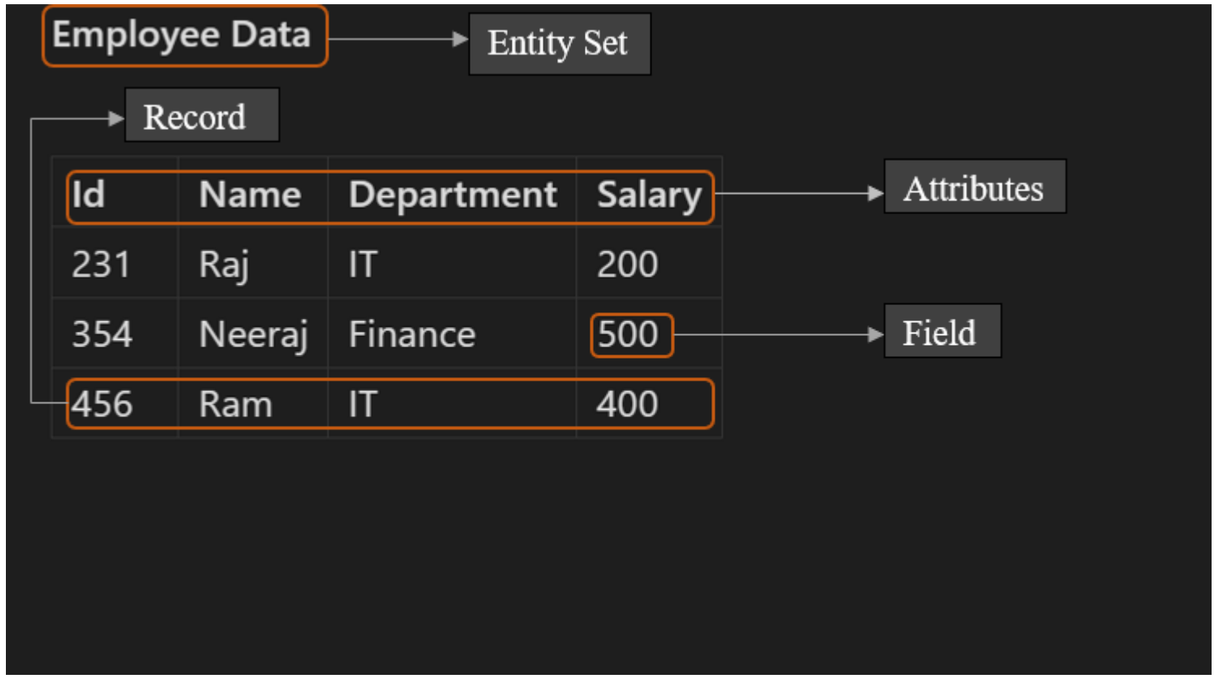
Introduction to Data Structures and Algorithm

Module Contents

Data and Information
Type of Data
Classification and Operation of Data Structure
Abstract Data Type (ADT)
Algorithm

Data -> *Source of Information*
Information -> Processed data / Meaningful data.



Entity Attributes

Field :- A field is single elementary unit of information representing an attribute of an entity.
Record :- A record is a collection of field values of an entity.
Entity Set :- Collection of records of all the entities.

Type of Data

1. Simple Data :- Represents simple value.
2. Complex Data :- Represents group of value. Example: Student={Roll no, Age, Marks, Address}

Implementation as per data

- To deal with the *Simple Data* we use primitive / built-in data type. Primitive data type has same pre-defined characteristics and properties
- To deal with complex data type we use non-primitive/user-defined data types.
  - In C/C++ we use `structure`.
  - In C++/Java we use `class`.

Data Structure

Data Structure refers to logical or mathematical model of organizing and storing data in a computer so that it can be accessed and used efficiently.

Operations on Data Structure

1. Traversing : Accessing or visiting each record exactly once so that certain item in the record may be processed.
2. Insertion : Adding new record to the structure.
3. Deletion : Removing a record from the structure.
4. Searching : Finding a location of the record with a given key value or finding the locations of all the records which satisfy one or more condition.
5. Sorting : Arranging the records in some logical order (e.g. alphabetically or in numerical order.)
6. Merging : Combining the records in two different sorted files into a single sorted file.

   Example for Data Structure Operation

An organization contains a membership file in which each record contains the following data for a given member:

Name, Address, Telephone Number, Age, Sex

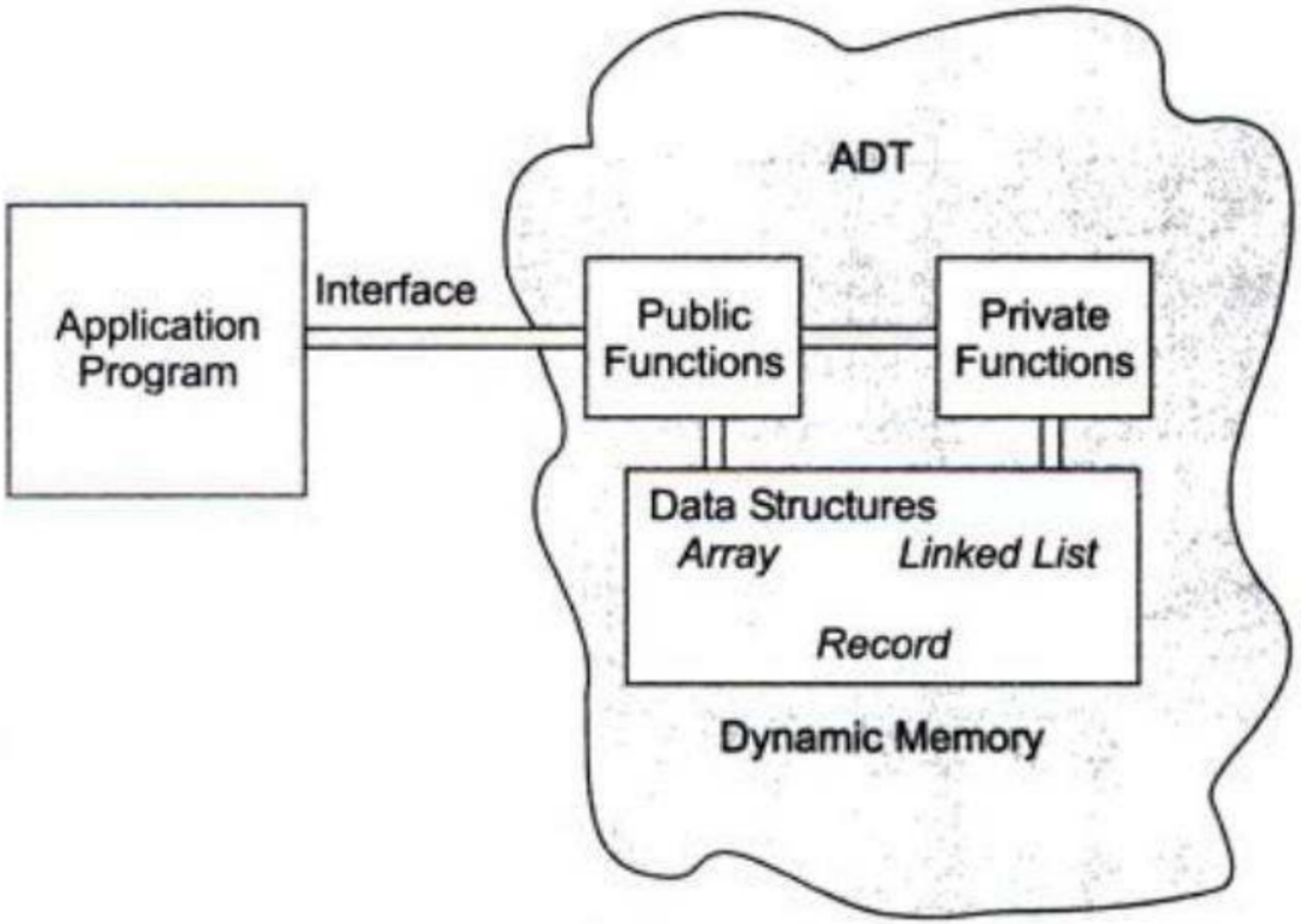
(a) Suppose the organization wants to announce a meeting through a mailing. Then one would traverse the file to obtain Name and Address for each member.
(b) Suppose one wants to find the names of all members living in a certain area. Again one would traverse the file to obtain the data.
(c) Suppose one wants to obtain Address for a given Name. Then one would search the file for the record containing Name.
(d) Suppose a new person joins the organization. Then one would insert his or her record into the file.
(e) Suppose a member dies. Then one would delete his or her record from the file.
(f) Suppose a member has moved and has a new address and telephone number. Given the name of the member, one would first need to search for the record in the file. Then one would perform the "update"—i.e., change items in the record with the new data.
(g) Suppose one wants to find the number of members 65 or older. Again one would traverse the file, counting such members.

Abstract Data Type (ADT)

Abstract Data Type (ADT)

Abstract data type refers to group of data members and associated member function together in a single unit.

There might be a situation when we need operations for our user-defined data type which have to be defined. These operations can be defined only as and when we require them. So, in order to simplify the process of solving problems, we can create data structures along with their operations, and such data structures that are not in-built are known as Abstract Data Type (ADT).

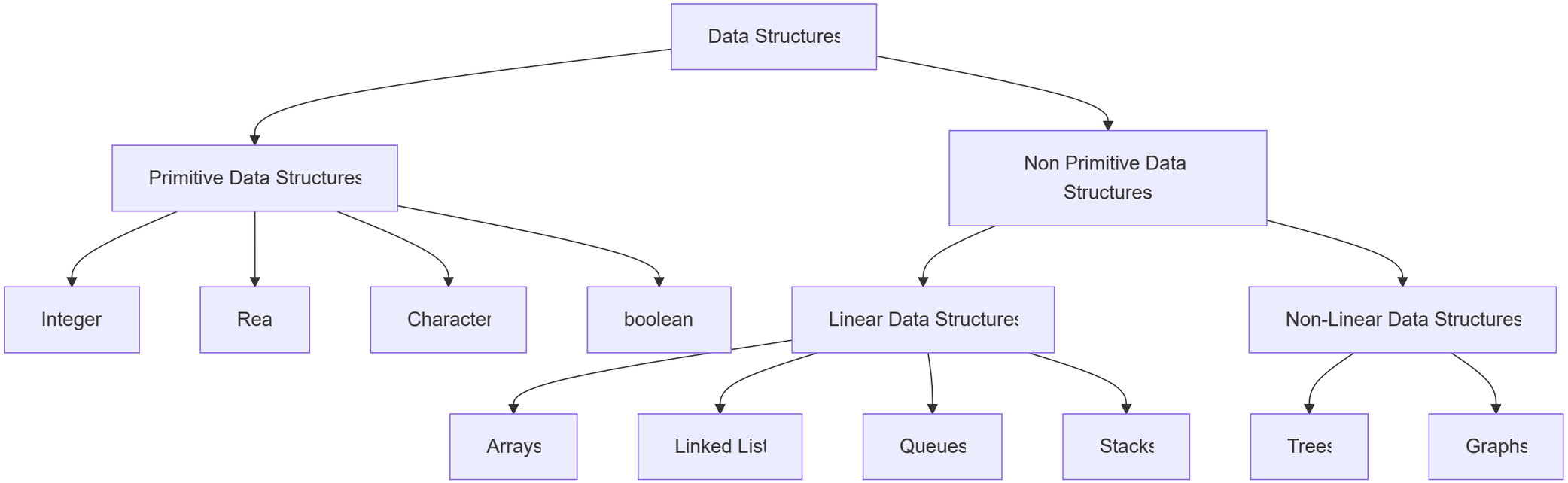


ADT Model

There are two different parts of ADT Model :-

1. Functions (public an private)
2. Data Structures
   Both contained within the ADT model and do not come with the scope of *Application Program*. Data structures are available to all ADTs functions and function may call any other function to accomplish the task. This means data structures and functions are in the same scope. Data is modifies or accessed through external application programing interface. The interface can access only public function to perform an operation by passing the parameters.

Classification of Data Structures



Algorithm

> **Algorithm**
>
> An algorithm is a set of finite and well-defined rules or instruction to perform a particular task.

Characteristics of Algorithm

1. Correctness : The algorithm should solve the problem accurately.
2. Efficiency : Algorithm should use time and resources optimally.
3. Clarity: The algorithm should be easy to understand and implement.
4. Definiteness: Each step of the algorithm should be precise and unambiguous, and the algorithm should not be open to multiple interpretations. For example, the number of times an algorithm should iterate must be precisely expressed.
5. Finiteness: The algorithm should end after a specific number of steps.

Efficiency of an Algorithm

One of the major characteristics of the algorithm is efficiency for processing the data. The time and space it uses are the two major measure of efficiency of an algorithm. The ***Complexity*** of an algorithm is the function which gives the running time or space in terms of the input size.

> **Time-Space Tradeoff**

Each of algorithm will involve a particular data structure. Accordingly we may not always able to use the most efficient algorithm, since the choice of data structure depends on many things, including type of data and the frequency with which various data operation are applied. Sometimes the choice of data structure involves a time-space tradeoff: by increasing the amount of space for storing the data, one may be able to reduce the time needed for processing the data or vice versa.